From Model Artifacts to Production APIs

Domain: Bioactivity Prediction of Chemical Compounds
This document serves as your technical roadmap for **Assignment 1**. We assume you already have a trained model artifact (e.g., bioactivity_model.joblib) that takes molecular descriptors or SMILES strings as input.

🛠️ Step 0: Engineering Environment Setup

Before writing code, ensure your environment is configured.

```
# Core serving and validation stack
pip install flask fastapi uvicorn joblib scikit-learn httpx streamlit
```

🏺 Step 2: The Legacy Approach (Flask)

Flask uses the **WSGI (Web Server Gateway Interface)**. It follows a synchronous, blocking pattern. In this implementation, you are the "Human Firewall"—you must manually validate everything before it touches your model.

📝 Flask Implementation Guidance

- **Manual Validation:** You cannot trust the user input. You must write if/else logic to check request.get_json().
- **The Global Load:** In Flask, we load the model once at the top of the script so it persists in memory.

Code Structure for flask_app.py:

```
import os
import joblib
from flask import Flask, request, jsonify

app = Flask(__name__)

# 1. Verification & Global Loading
MODEL_PATH = 'bioactivity_model.joblib'
if not os.path.exists(MODEL_PATH):
    raise FileNotFoundError(f"Missing model artifact at {MODEL_PATH}")

# Loading globally (Legacy Pattern)
model = joblib.load(MODEL_PATH)

@app.route('/predict', methods=['POST'])
def predict():
```

```
    # 2. Manual Defensive Layer (The Firewall)
    data = request.get_json()
    if not data or 'smiles' not in data:
        return jsonify({"error": "Bad Request: 'smiles' field is required"}), 400

    smiles_string = data['smiles']

    # 3. Manual Type & Content Validation
    if not isinstance(smiles_string, str) or len(smiles_string.strip()) < 3:
        return jsonify({"error": "Validation Error: 'smiles' must be a valid string"}), 400

    # 4. Synchronous Inference (Blocks this worker thread)
    # The model expects a list/array of SMILES strings
    prediction = model.predict([smiles_string])[0]
    result = "Active" if prediction == 1 else "Inactive"

    return jsonify({
        "smiles": smiles_string,
        "activity": result,
        "engine": "Flask/WSGI"
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

⚡ Step 3: The Modern Approach (FastAPI)

FastAPI uses **ASGI (Asynchronous Server Gateway Interface)**. It automates the "Firewall" logic using **Pydantic** and manages the model lifecycle using the **Lifespan** pattern.

📝 FastAPI Implementation Guidance

- **The Contract:** You define a BaseModel. FastAPI automatically returns a 422 Unprocessable Entity error if the SMILES string is malformed or missing.
- **Lifespan Pattern:** This is the professional way to load heavy assets. The model is loaded once on startup and stored in app.state.
- **Background Tasks:** You can trigger side-effects (like logging) without making the user wait for them to finish.

Code Structure for fastapi_app.py:

```
import joblib
import time
from fastapi import FastAPI, Request, HTTPException, BackgroundTasks
```

```
from pydantic import BaseModel, Field
from contextlib import asynccontextmanager

# 1. Define the Data Contract (SMILES Input)
class BioactivityRequest(BaseModel):
    # Field constraints handle validation automatically
    smiles: str = Field(..., min_length=3, max_length=1000,
example="CN1C=NC2=C1C(=O)N(C(=O)N2C)C")

# 2. Manage Resource Lifecycle (Lifespan)
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: Load model into app state once
    try:
        app.state.model = joblib.load('bioactivity_model.joblib')
        print("✅ Bioactivity model loaded into memory.")
    except Exception as e:
        print(f"❌ Load Error: {e}")
        raise
    yield
    # Shutdown: Cleanup
    app.state.model = None

app = FastAPI(title="Compound Bioactivity API", lifespan=lifespan)

def log_prediction(smiles: str, result: str):
    """Simulates a background task like logging to a database."""
    with open("prediction_log.txt", "a") as f:
        f.write(f"Compound: {smiles} | Result: {result}\n")

@app.post("/predict")
async def predict(request: Request, body: BioactivityRequest, bg_tasks: BackgroundTasks):
    model = request.app.state.model

    # 3. Asynchronous Inference Workflow
    # Prediction logic
    prediction = model.predict([body.smiles])[0]
    activity = "Active" if prediction == 1 else "Inactive"

    # 4. Trigger Background Task (User doesn't wait for this)
    bg_tasks.add_task(log_prediction, body.smiles, activity)

    return {
```

```
    "smiles": body.smiles,
    "activity": activity,
    "engine": "FastAPI/ASGI"
}
```

🎨 Step 4: Re-integrating with Streamlit (The Frontend)

Now that your model is a standalone service, your Streamlit app no longer needs to load the heavy .joblib file itself. Instead, it acts as a "Client" that talks to your API.

📝 Integration Guidance

- **Decoupling:** Your Streamlit app stays lightweight. If you update the model weights in the API, the Streamlit app doesn't even need to be restarted.
- **Request Handling:** Use the requests library to send the SMILES string to http://localhost:8000/predict.

💡 **Pro-Tip: Localhost vs. Remote Servers** When working locally, localhost refers to your own computer. You must have **two** terminal windows open:

1. One running your API (uvicorn fastapi_app:app ...).
2. One running your Streamlit app (streamlit run streamlit_app.py).

In a real production environment, you would replace localhost with the public IP or domain of your API server (e.g., https://api.biotech-firm.com/predict).

Streamlit Client Code (streamlit_app.py):

```
import streamlit as st
import requests

st.title("Bioactivity Predictor")
smiles = st.text_input("Enter SMILES String", "CN1C=NC2=C1C(=O)N(C(=O)N2C)C")

if st.button("Predict"):
    # Send request to our FastAPI server
    payload = {"smiles": smiles}
    try:
        response = requests.post("http://localhost:8000/predict", json=payload)

        if response.status_code == 200:
            res = response.json()
            st.success(f"Prediction: {res['activity']}")
            st.info(f"Backend Engine: {res['engine']}")
        else:
            st.error(f"API Error: {response.text}")
```

```
    except Exception as e:
        st.error(f"Could not connect to API: {e}")
```

📈 Step 5: Comparative Stress Testing

To truly understand why the industry is moving toward FastAPI, observe the **"Waiter Problem"** discussed in the slides. We will simulate 50 concurrent chemical submissions.

Benchmark script (stress_test.py):

```
import asyncio
import httpx
import time

async def benchmark(name, url, concurrency=50):
    async with httpx.AsyncClient() as client:
        start = time.time()
        # Simulate 50 concurrent researchers submitting compounds
        tasks = [client.post(url, json={"smiles": "CC(=O)OC1=CC=CC=C1C(=O)O"}, timeout=15)
for _ in range(concurrency)]
        results = await asyncio.gather(*tasks, return_exceptions=True)

        success = [r for r in results if not isinstance(r, Exception) and r.status_code == 200]
        print(f"--- {name} Benchmark ---")
        print(f"Total time for {concurrency} requests: {time.time() - start:.2f}s")
        print(f"Successes: {len(success)}/50\n")

# Run locally:
# asyncio.run(benchmark("Flask", "http://localhost:5000/predict"))
# asyncio.run(benchmark("FastAPI", "http://localhost:8000/predict"))
```